

Objectifs généraux

Un système de fichiers est une méthode d'organisation, de stockage et de récupération de données sur un support de stockage. Dans un système d'exploitation réel, cette organisation repose sur plusieurs niveaux d'abstraction : le matériel fournit une suite de blocs adressables, le système de fichiers définit une structure logique permettant d'interpréter ces blocs, puis les mécanismes d'accès aux fichiers utilisent ces structures pour retrouver les données associées à un nom, un identifiant ou un descripteur.

Dans ce TP, vous allez construire progressivement un **Virtual File System (VFS)** dont le support de stockage est constitué d'un simple tableau Java `byte[]`. Le tableau représente une mémoire brute : il ne possède intrinsèquement aucune notion de fichier, de répertoire, d'inode, de bloc libre ou de métadonnée. Toutes ces notions devront donc être construites explicitement par votre programme. L'objectif n'est pas uniquement d'obtenir un programme qui fonctionne, mais de comprendre comment une structure logicielle de haut niveau peut être reconstruite à partir d'un espace mémoire initialement dépourvu de structure.

Le VFS constitue ainsi une forme de laboratoire expérimental permettant d'observer les mécanismes qui, dans un système réel, sont masqués par le système d'exploitation. Vous manipulerez directement les octets, les offsets, les blocs, les structures binaires, les tables de métadonnées et les mécanismes d'allocation. Le tableau `byte[]` peut être considéré comme une abstraction extrêmement simplifiée d'un disque ou d'une image disque : sa taille est fixe, ses positions sont adressables, et les données qui y sont placées ne prennent un sens que par convention.

Une difficulté centrale du TP vient précisément de cette absence de typage dans la mémoire simulée. Un `byte[]` ne sait pas qu'un groupe de quatre octets représente un `int`, qu'un groupe de huit octets représente un timestamp, qu'une zone particulière correspond au bitmap ou qu'une autre zone correspond à un inode. C'est votre architecture qui impose cette interprétation. Cette séparation entre **représentation physique** et **interprétation logique** constitue une idée fondamentale des systèmes informatiques bas niveau.

Le TP est donc organisé autour d'une progression volontairement ascendante :

1. construire les primitives de sérialisation binaire ;
2. construire une mémoire virtuelle structurée ;
3. gérer l'allocation des blocs avec un bitmap ;
4. définir et sérialiser les inodes ;
5. créer des fichiers ;
6. écrire et relire leurs données ;
7. observer les effets de l'allocation et de la fragmentation ;
8. tester le système avec des données produites à l'extérieur du VFS.

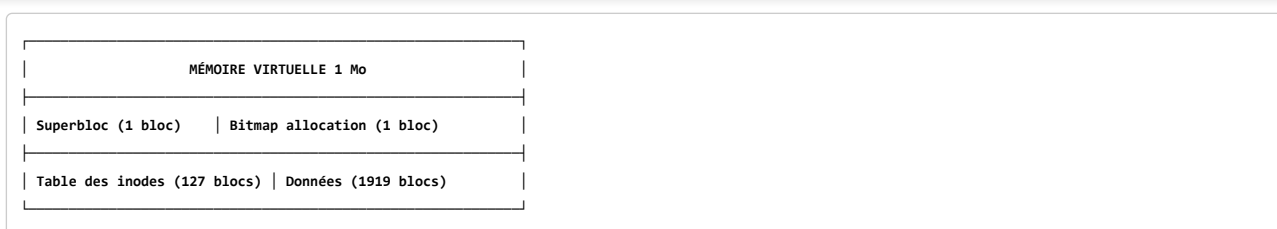
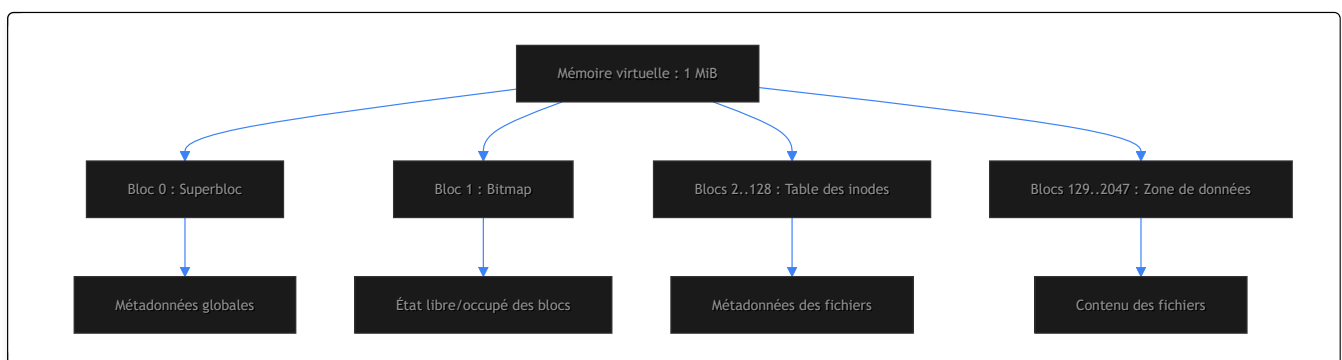
L'ensemble forme une chaîne de dépendances : une erreur introduite au niveau de la sérialisation peut contaminer le superbloc, puis les inodes, puis les opérations sur les fichiers. Il est donc demandé de respecter l'ordre des étapes et de valider chaque couche avant de construire la suivante.

Vue générale de l'architecture

Avant de commencer l'implémentation, il est nécessaire de comprendre la structure globale du système. Le VFS repose sur une mémoire de taille fixe de **1 MiB**. Cette mémoire est divisée en blocs de taille fixe de **512** octets. Certains blocs sont réservés aux structures internes du système de fichiers tandis que les blocs restants constituent l'espace disponible pour les données utilisateur. La relation fondamentale est :

$$\text{Nombre de blocs} = \frac{\text{Taille totale de la mémoire}}{\text{Taille d'un bloc}} = \frac{1024 \times 1024}{512} = 2048$$

Le système possède donc exactement **2048** blocs adressables, numérotés de 0 à 2047. La mémoire est ensuite découpée en différentes régions. Cette organisation est appelée le **layout du système de fichiers**. Les figures ci-dessous illustre l'organisation générale de notre système :



La séparation entre métadonnées et données est une idée fondamentale. Le contenu d'un fichier ne doit pas être confondu avec les informations permettant de retrouver ce contenu. Un inode décrit par exemple la taille d'un fichier et les blocs dans lesquels ses données sont stockées, tandis que les blocs de données contiennent effectivement les octets du fichier.

CONTRAINTE MÉMOIRE CRITIQUE : Contrairement aux exemples classiques qui utilisent `HashMap`, `ArrayList`, `BitSet` de Java, **TOUTES** nos structures doivent être stockées dans le tableau `byte[]` de 1Mo. Aucune allocation mémoire externe n'est autorisée. Cela impose une gestion manuelle des offsets et la sérialisation/désérialisation de toutes les structures.

Dit autrement : la mémoire est géré comme un classeur

1. Les pages = Blocs de données (512 octets chacun)
 - Votre classeur a exactement 2048 pages de 512 octets chacune
 - Total = 1 Mo de mémoire
2. L'index = Table des Inodes
 - Comme l'index d'un livre : "Le document X se trouve aux pages 15, 16, 17"
 - Chaque fichier a une "fiche" (inode) qui dit où il est stocké
3. Le plan d'occupation = Bitmap
 - Une liste qui dit : "page 1 : occupée, page 2 : libre, page 3 : occupée..."

Attention : Notre système de fichiers virtuel ne gère pas la persistance automatique. Toutes les données sont perdues à la fin du programme sauf si explicitement sauvegardées. C'est une limitation volontaire pour se concentrer sur les algorithmes de base.

Concepts fondamentaux

Blocs de données : L'unité de base d'allocation dans notre système de fichiers. Nous utiliserons des blocs de 512 octets, similaires aux secteurs d'un disque dur traditionnel. Avec 1 Mo de mémoire disponible, nous disposons de 2048 blocs ($1\,048\,576 \div 512 = 2048$).

Inodes (Index Nodes) : Structure de données qui contient les métadonnées d'un fichier : taille, permissions, horodatage, et pointeurs vers les blocs de données. Chaque fichier et répertoire possède un inode unique identifié par un numéro.

Table d'allocation : Bitmap qui indique quels blocs sont libres ou occupés. Essentielle pour éviter les conflits lors de l'allocation de nouveaux blocs.

Répertoires : Fichiers spéciaux contenant une liste d'entrées, chaque entrée associant un nom de fichier à un numéro d'inode.

Étape 1 : Initialisation et Git

Un projet manipulant un système de fichiers virtuel produit plusieurs catégories de fichiers qui n'ont pas toutes leur place dans le dépôt Git. Les fichiers Java contenant le code source constituent l'artefact principal du projet, tandis que les fichiers compilés, archives et images disque représentent des artefacts générés.

Cette distinction est importante dans un projet logiciel sérieux. Un fichier `.class` dépend de l'environnement de compilation et peut être régénéré à partir du `.java`. De même, une image disque virtuelle peut être générée à partir du programme. Versionner systématiquement ces artefacts peut introduire du bruit dans l'historique et, plus fondamentalement, masquer la distinction entre **source** et **résultat d'exécution**.

Le système de fichiers que vous allez construire pourra également être sauvegardé dans un fichier image. Cette image représentera alors l'état binaire du système à un instant donné. Elle peut être utile pour des tests ou pour l'inspection du contenu brut, mais elle ne doit pas être considérée comme le code source du projet.

Instructions

À la racine du dépôt `OS_Nom1_Nom2/`, créez le `.gitignore` global :

```
echo "*.class" >> .gitignore
echo "*.jar" >> .gitignore
```

Créez ensuite le dossier du TP :

```
mkdir SystemeFichiersVirtuel
cd SystemeFichiersVirtuel
echo "filesystem.img" >> .gitignore
```

Vérifiez ensuite l'état du dépôt :

```
git status
```

Un bon dépôt logiciel doit permettre à un autre développeur de reconstruire les artefacts générés à partir des sources versionnées. Cette propriété est particulièrement importante ici, car le système de fichiers manipulé par le programme est lui-même un artefact binaire généré en mémoire.

Travail demandé

1. Vérifiez que les fichiers `.class` ne sont pas suivis par Git.
2. Vérifiez que `filesystem.img` est ignoré.
3. Effectuez un premier commit contenant uniquement les sources et fichiers nécessaires au TP.
4. Conservez un historique Git suffisamment régulier pour permettre de distinguer les différentes étapes d'implémentation.

Étape 2 : Sérialisations simples

La première difficulté consiste à transformer des valeurs Java typées en une représentation constituée uniquement d'octets. Java distingue explicitement les types `byte`, `short`, `int` et `long`. La mémoire simulée du VFS, elle, ne connaît que `byte[]`. Il faut donc définir une convention de représentation.

- Un `int` Java occupe **32** bits, soit $32/8 = 4$ octets
- Un `short` occupe **16** bits, soit $16/8 = 2$ octets
- Un `long` occupe **64** bits, soit $64/8 = 8$ octets

Vous devez utiliser une représentation **big-endian** : l'octet de poids fort est placé à l'adresse la plus faible.

Pour une valeur `V = 0xF0A1B2E3` la représentation mémoire attendue est :

```
offset + 0 : F0
offset + 1 : A1
```

```
offset + 2 : B2
offset + 3 : E3
```

Cette convention doit être déterministe. Si deux fonctions d'écriture différentes utilisent des conventions différentes, les métadonnées du système de fichiers deviendront impossibles à interpréter correctement.

Découpage d'un entier

Pour extraire les différents octets d'un `int`, vous devez utiliser les décalages binaires.

Conceptuellement :

$$b_3 = V \& 0xFF$$

La lecture effectue l'opération inverse.

En Java, `byte` est signé. Une valeur mémoire telle que `0xF0` peut donc apparaître comme une valeur négative lorsqu'elle est interprétée comme un `byte`. Lorsqu'un octet doit être utilisé comme quantité non signée avant une recombinaison, le masquage `& 0xFF` est essentiel.

Cette question n'est pas spécifique au VFS. Elle apparaît dans de nombreux contextes de programmation système : protocoles réseau, formats binaires, fichiers structurés, compression, cryptographie, sérialisation et communication avec du matériel.

Squelette `Utils.java`

```
public class Utils {

    public static int writeInt(byte[] memory, int offset, int value) {
        // TODO: Écrire les 4 octets de 'value' dans 'memory'
        // à partir de 'offset', en big-endian.
        return 4;
    }

    public static int readInt(byte[] memory, int offset) {
        // TODO: Reconstituer le int sur 4 octets.
        return 0;
    }

    public static int writeShort(byte[] memory, int offset, short value) {
        // TODO: Écrire les 2 octets de 'value'.
        return 2;
    }

    public static short readShort(byte[] memory, int offset) {
        // TODO: Lire le short sur 2 octets.
        return 0;
    }
}
```

► Tests obligatoires

Testez également une valeur contenant des bits de poids fort à 1, par exemple `0x80000000`, afin de vérifier que votre traitement du complément à deux ne provoque pas d'erreur.

Exécution :

```
javac *.java
java -ea TestRunner
```

L'option `-ea` active les assertions Java.

Étape 3 : Long et chaîne

Sérialisation d'un `long`

Les métadonnées d'un système de fichiers contiennent fréquemment des valeurs temporelles. Dans ce TP, les temps sont représentés par la valeur retournée par :

```
System.currentTimeMillis()
```

Cette valeur nécessite un `long`, donc 64 bits : $64/8 = 8$ octets. Le principe est identique à celui du `int`, mais le domaine de représentation est plus large. La mémoire doit contenir les huit octets dans un ordre déterministe. Le principe général peut être exprimé comme

$$V = \sum_{i=0}^7 b_i \times 2^{8(7-i)}$$

où chaque b_i représente un octet de la représentation binaire.

Sérialisation des chaînes

Les chaînes constituent un problème légèrement différent. Une `String` Java est une structure abstraite représentant du texte, alors que le système de fichiers manipule des octets.

Il faut donc définir une représentation textuelle sous forme d'octets. Dans le cadre de ce TP, vous utiliserez :

```
str.getBytes()
```

Une chaîne occupe alors un nombre variable d'octets, mais les structures du système de fichiers utilisent souvent des champs de taille fixe. Il faut donc associer une longueur logique variable à une zone physique de taille déterminée. Si un champ possède `maxLength = 16`, la zone réservée contient exactement **16** octets, même si la chaîne ne contient que quatre caractères.

Par exemple :

```
"MYFS"
```

peut être représenté dans une zone de 16 octets comme :

```
4D 59 46 53 00 00 00 00 00 00 00 00 00 00 00 00
```

Le caractère nul joue ici le rôle de marqueur de fin de chaîne.

Le remplissage par zéro est une opération importante. Une mémoire peut avoir été utilisée précédemment par une autre donnée. Si une nouvelle chaîne plus courte est écrite sans nettoyer le reste de sa zone, les anciens octets restent physiquement présents. Le système logique peut alors donner l'impression que les données sont correctes tout en conservant des informations résiduelles dans la représentation brute.

Squelette

Complétez `Utils.java` :

```
public static int writeLong(byte[] memory, int offset, long value) {
    // TODO: Écrire les 8 octets du long en big-endian.
    return 8;
}

public static long readLong(byte[] memory, int offset) {
    // TODO: Reconstituer le long.
    return 0L;
}

public static int writeString(
    byte[] memory,
    int offset,
    String str,
    int maxLength) {

    // TODO:
    // 1. Convertir la chaîne en octets.
    // 2. Copier les octets sans dépasser maxLength.
    // 3. Nettoyer le reste de la zone avec des zéros.

    return maxLength;
}

public static String readString(
    byte[] memory,
    int offset,
    int maxLength) {

    // TODO:
    // Lire jusqu'au premier octet nul
    // ou jusqu'à maxLength.

    return "";
}
```

► Tests obligatoires

Étape 4 : Architecture mémoire

Modèle physique

Le système de fichiers virtuel possède $M = 1024 \times 1024 = 1\,048\,576$ octets de mémoire. Chaque bloc possède $B = 512$ octets. Le nombre de blocs est donc :

$$N = \frac{M}{B} = 2048$$

Chaque adresse physique du VFS peut être considérée comme un offset dans le tableau `byte[]`. Pour un bloc de numéro i , son adresse de début est :

$$\text{offset}(i) = i \times B$$

Cette formule sera utilisée de manière répétée dans la suite du TP.

Layout

La mémoire est organisée comme suit :



Le bitmap contient un bit par bloc :

$$2048 \text{ blocs} \times 1 \text{ bit} = 2048 \text{ bits}$$

soit :

$$2048 / 8 = 256 \text{ octets}$$

Les blocs 0 à 128 sont réservés au système. La zone de données commence donc au bloc 129. Le nombre de blocs de données disponibles est $2048 - 129 = 1919$

Les numéros de blocs et les offsets mémoire sont deux notions liées mais distinctes. Le bloc 129 commence à l'offset $129 * 512$, et non à l'offset 129. Une confusion entre ces deux unités produit des corruptions mémoire difficiles à diagnostiquer.

Squelette `MemoryManager.java`

```

import java.io.*;

public class MemoryManager {

    public static final int BLOCK_SIZE = 512;
    public static final int TOTAL_MEMORY = 1024 * 1024;
    public static final int NUM_BLOCKS =
        TOTAL_MEMORY / BLOCK_SIZE;

    public static final int SUPERBLOCK_OFFSET = 0;
    public static final int BITMAP_OFFSET = BLOCK_SIZE;
    public static final int INODE_TABLE_OFFSET =
        2 * BLOCK_SIZE;
    public static final int DATA_OFFSET =
        129 * BLOCK_SIZE;

    public static final int INODE_SIZE = 128;

    public static final int INODE_TABLE_SIZE =
        DATA_OFFSET - INODE_TABLE_OFFSET;

    public static final int MAX_INODES =
        INODE_TABLE_SIZE / INODE_SIZE;

```

```

private byte[] memory;

public MemoryManager() {
    this.memory = new byte[TOTAL_MEMORY];
    initializeFilesystem();
}

private void initializeFilesystem() {
    writeSuperblock();

    // TODO:
    // Réserver les blocs système 0 à 128.
}

private void writeSuperblock() {
    // TODO:
    // Utiliser Uutils pour écrire les métadonnées.

    Uutils.writeString(
        memory,
        SUPERBLOCK_OFFSET,
        "MYVFS1.0",
        16);

    Uutils.writeInt(
        memory,
        SUPERBLOCK_OFFSET + 16,
        BLOCK_SIZE);

    Uutils.writeInt(
        memory,
        SUPERBLOCK_OFFSET + 20,
        TOTAL_MEMORY);

    Uutils.writeInt(
        memory,
        SUPERBLOCK_OFFSET + 24,
        NUM_BLOCKS);

    Uutils.writeInt(
        memory,
        SUPERBLOCK_OFFSET + 28,
        MAX_INODES);
}

public byte[] getFilesystemMemory() {
    return memory;
}
}

```

► Tests obligatoires

Étape 5 : Gestion du bitmap

Le système doit maintenant savoir quels blocs sont libres et quels blocs sont occupés. Une solution naïve consisterait à réserver un entier ou un booléen pour chaque bloc. Une telle représentation serait fonctionnelle mais relativement coûteuse. Le bitmap utilise une représentation plus compacte : **un seul bit par bloc**.

Pour N blocs, le bitmap nécessite $\frac{N}{8}$ octets. Pour 2048 blocs : $\frac{2048}{8} = 256$ octets.

Cette représentation est typique des systèmes bas niveau : lorsqu'un grand nombre d'éléments ne possède qu'un état binaire, les bits permettent de représenter l'état de manière extrêmement compacte.

Pour un bloc N , l'indice de l'octet est :

$$byteIndex = \left\lfloor \frac{N}{8} \right\rfloor$$

et la position du bit :

$$bitPosition = N \bmod 8$$

L'adresse mémoire correspondante est :

$$bitmapOffset = BITMAP_OFFSET + byteIndex$$

Exemple

Pour le bloc 129 :

$$129/8 = 16$$

avec division entière, et :

$$129 \bmod 8 = 1$$

Le bloc 129 correspond donc au bit 1 de l'octet situé à :

$$512 + 16 = 528$$

Le bit correspondant vaut :

$$1 \ll 1 = 0b00000010$$

Le bitmap n'enregistre pas directement la taille d'un fichier ni son contenu. Il indique uniquement l'état d'allocation du bloc. Les informations permettant de savoir quels blocs appartiennent à un fichier seront stockées dans l'inode.

Opérations binaires

Pour positionner un bit :

```
octet |= masque
```

Pour effacer un bit :

```
octet &= ~masque
```

Pour lire un bit, vous devez extraire sa valeur sans être perturbé par le signe du `byte`.

Le bitmap doit également respecter un invariant fondamental :

un bloc système réservé ne doit jamais être retourné comme bloc de données libre.

L'allocation devra donc commencer au bloc 129.

Squelette

```
public boolean setBlockUsed(int blockNumber, boolean used) {
    if (blockNumber < 0 ||
        blockNumber >= NUM_BLOCKS) {
        return false;
    }

    int byteIndex = blockNumber / 8;
    int bitPosition = blockNumber % 8;
    int offset = BITMAP_OFFSET + byteIndex;

    if (used) {
        // TODO:
        // Positionner le bit à 1.
    } else {
        // TODO:
        // Positionner le bit à 0.
    }

    return true;
}

public int isBlockUsed(int blockNumber) {

    if (blockNumber < 0 ||
        blockNumber >= NUM_BLOCKS) {
        return -1;
    }

    // TODO:
    // Calculer byteIndex.
    // Calculer bitPosition.
    // Lire le bit.

    return -1;
}

public int allocateBlock() {

    // TODO:
    // Parcourir les blocs de données :
    // 129 .. NUM_BLOCKS - 1.
    //
    // Retourner le premier bloc libre.
```

```
// Le marquer immédiatement comme utilisé.

return -1;
}
```

▼ Tests obligatoires

```
public static void testStep5() {
    System.out.println("=== TEST ÉTAPE 5 : Bitmap et Allocation ===");

    MemoryManager mm = new MemoryManager();

    assert mm.setBlockUsed(130, true) :
        "setBlockUsed doit réussir";

    assert mm.isBlockUsed(130) == 1 :
        "Le bloc 130 doit être occupé";

    assert mm.setBlockUsed(130, false) :
        "La libération doit réussir";

    assert mm.isBlockUsed(130) == 0 :
        "Le bloc 130 doit être libre";

    mm.setBlockUsed(129, true);

    int bitmapOffset =
        MemoryManager.BITMAP_OFFSET + (129 / 8);

    assert (mm.getFilesystemMemory()[bitmapOffset]
        & 0xFF) == 0x02 :
        "Le bit du bloc 129 est incorrect";

    mm.setBlockUsed(130, true);

    assert (mm.getFilesystemMemory()[bitmapOffset]
        & 0xFF) == 0x06 :
        "Les bits 129 et 130 sont incorrects";

    mm.setBlockUsed(130, false);

    assert (mm.getFilesystemMemory()[bitmapOffset]
        & 0xFF) == 0x02 :
        "La libération du bloc 130 est incorrecte";

    MemoryManager mm2 = new MemoryManager();

    int first = mm2.allocateBlock();
    int second = mm2.allocateBlock();

    assert first == 129 :
        "Le premier bloc de données doit être 129";

    assert second == 130 :
        "Le second bloc de données doit être 130";

    assert mm2.isBlockUsed(129) == 1;
    assert mm2.isBlockUsed(130) == 1;

    assert mm2.isBlockUsed(-1) == -1 :
        "Un bloc négatif doit être refusé";

    assert mm2.isBlockUsed(
        MemoryManager.NUM_BLOCKS) == -1 :
        "Un bloc hors limites doit être refusé";

    System.out.println("[OK] Étape 5 validée !");
}
```

Étape 6 : Layout des inodes

Un fichier ne peut pas être identifié uniquement par ses données. Le système doit également connaître sa taille, son type, ses dates, ses permissions et les blocs dans lesquels ses données sont stockées. Toutes ces informations constituent les **métadonnées** du fichier.

Dans notre VFS, elles sont regroupées dans une structure de taille fixe appelée **inode**. Chaque inode possède exactement **128 octets**. Cette structuration est rigide ! Elle est défini à la conception : c'est come ça et pas autrement. Jusqu'a ce que l'on change de version ou d'implémentation : mais attention a la rétro compatibilité ...

La table commence à : **$INODE_TABLE_OFFSET = 1024$** c'est donc le secteur numéro 2 si un secteur a une taille de 512. L'offset de l'inode **N** est donc :
 $Offset(N) = INODE_TABLE_OFFSET + N \times INODE_SIZE$

Donc avec **$INODE_SIZE = 128$** on trouve l'inode **4** via la relation :

$$Offset(4) = 1024 + (4 \times 128)$$

Cette formule doit être comprise comme une fonction d'adressage, un calcul de position. Une erreur d'un seul octet dans cette formule peut faire lire un inode à l'intérieur de la structure précédente ou suivante. Donc avec un décalage de l'information ce qui implique des résultats catastrophiques et incohérent. C'est au programmeur de garantir la robustesse.

Structure binaire

L'inode est organisé ainsi :

Offset relatif	Taille	Champ
0..3	4	Numéro d'inode
4..7	4	Type
8..11	4	Taille
12..19	8	Création
20..27	8	Modification
28..67	40	10 pointeurs directs
68..71	4	Pointeur indirect
72..73	2	Permissions
74..77	4	Nombre de liens

Les octets restants de la zone de 128 octets restent disponibles pour l'évolution future du format. Exemple : on veut implémenter une double indirection !

La taille fixe de l'inode simplifie considérablement l'adressage. Le système n'a pas besoin de parcourir une structure de taille variable pour trouver l'inode **N** : il suffit d'appliquer une multiplication et une addition. Cette propriété est fondamentale dans les structures de données destinées à être directement adressables sur disque. C'est beaucoup plus rapide ! Au prix d'une petite perte de mémoire.

Squelette

```
public class Inode {

    private MemoryManager memoryManager;
    private int inodeNumber;

    public static final int INODE_SIZE = 128;
    public static final int DIRECT_POINTERS = 10;

    public Inode(
        MemoryManager memoryManager,
        int inodeNumber) {

        this.memoryManager = memoryManager;
        this.inodeNumber = inodeNumber;
    }

    public int getInodeOffset() {
        // TODO:
        // Calculer l'offset exact de l'inode.
        return 0;
    }

    public int getFileType() {
        // TODO:
        // Lire le type à offset + 4.
        return 0;
    }

    public int getFileSize() {
        // TODO:
        // Lire la taille à offset + 8.
        return 0;
    }

    public int[] getDirectPointers() {

        byte[] memory =
            memoryManager.getFileSystemMemory();

        int[] pointers =
            new int[DIRECT_POINTERS];
    }
}
```

```
// TODO:
// Lire les 10 pointeurs directs.

return pointers;
}
}
```

► Tests obligatoires

Étape 7 : Sérialisation d'un inode

L'étape précédente définissait l'emplacement des informations. Cette étape consiste à transformer l'ensemble des champs logiques de l'inode en représentation binaire. Le principe général est celui d'un **format binaire structuré** : chaque champ possède un offset et une taille déterminés. Globalement la même histoire depuis le début. C'est un gros tableau de byte et on gère des positions dedans pour structurer et retrouver facilement (Ou ? Comment ? Quelle taille ?).

On peut représenter l'écriture comme une fonction :

$$S : \text{Inode logique} \rightarrow \text{suite d'octets}$$

On parle aussi de sérialisation.

et la lecture comme une fonction inverse :

$$D : \text{suite d'octets} \rightarrow \text{Inode logique}$$

On parle alors de désérialisation.

L'objectif est bien sûr une application "bijective" et donc que :

$$D(S(x)) = x$$

pour les valeurs valides du modèle. C'est potentiellement un très bon test unitaire à faire. On génère des données aléatoires, par exemple suivant une distribution uniforme et on vérifie exhaustivement que toutes ces combinaisons soient équivalentes après sauvegarde et rechargement. Cette propriété est appelée une propriété de réversibilité de la sérialisation. Elle est centrale pour toute structure persistante : écrire puis relire une structure doit permettre de reconstruire la même information logique.

Ordre d'écriture

Les champs doivent être écrits dans l'ordre imposé par le format, donc ici il vous est imposé :

1. numéro d'inode ;
2. type ;
3. taille ;
4. date de création ;
5. date de modification ;
6. dix pointeurs directs ;
7. pointeur indirect ;
8. permissions ;
9. nombre de liens.

Vous devez gérer explicitement les offsets. Il est recommandé de raisonner en termes d'un curseur évoluant dans la structure :

$$offset_{i+1} = offset_i + taille(champ_i)$$

Cette méthode limite les erreurs de calcul manuel. C'est donc quelque chose qui pouvait être utilisé plus tôt. Vous aurez certainement observé la relation directe avec la lecture et l'écriture de fichier et la notion de curseur : c'est un principe équivalent.

Squelette

```
public void writeToMemory(
    int fileType,
    int fileSize,
    long creationTime,
    long modificationTime,
    int[] directPointers,
    int indirectPointer,
    short permissions,
    int linkCount) {

    byte[] memory =
        memoryManager.getFilesystemMemory();

    int offset = getInodeOffset();

    // TODO:
    // 1. Numéro d'inode
    // 2. Type
    // 3. Taille
    // 4. Création
    // 5. Modification
    // 6. 10 pointeurs directs
    // 7. Pointeur indirect
    // 8. Permissions
```

```
// 9. Nombre de liens
}
```

► Tests obligatoires

Nous disposons désormais des deux mécanismes fondamentaux permettant de représenter les informations du système :

- les octets peuvent être sérialisés
- un inode possède une représentation physique déterminée

Il faut maintenant créer une abstraction correspondant à un fichier. La création d'un fichier ne signifie pas nécessairement écrire immédiatement des données. Elle consiste d'abord à réserver une structure de métadonnées capable de représenter ce fichier.

Un fichier nouvellement créé possède donc typiquement :

- un inode ;
- un type indiquant qu'il s'agit d'un fichier ;
- une taille nulle ;
- aucun bloc de données associé ;
- des valeurs initiales pour ses autres métadonnées.

Cette distinction entre **création du fichier** et **allocation des données** est importante. Un fichier vide existe logiquement même s'il ne possède aucun bloc de données utilisateur. Il prend évidemment un peu de place / la taille d'un inode. Mais on ne le voit pas forcément, car cette taille d'inode est déjà réservé.

Allocation d'un inode

La table contient $MAX_INODES = \frac{INODE_TABLE_SIZE}{INODE_SIZE}$ inodes.

L'allocation consiste à parcourir cette table et à identifier un inode libre. Vous devrez définir, à partir du format fourni, comment reconnaître un inode disponible. Linux autorise notamment l'utilisation du type 0 ou d'un numéro d'inode invalide comme indicateur de disponibilité (eg -1 ou 0).

Ne confondez pas l'allocation d'un inode et l'allocation d'un bloc de données. Un inode décrit un fichier ; un bloc contient potentiellement une partie de son contenu. Les deux ressources sont gérées par des structures différentes.

Squelette

```
import java.util.*;

public class VirtualFileSystem {

    private MemoryManager memoryManager;

    public VirtualFileSystem() {
        this.memoryManager =
            new MemoryManager();
    }

    private int allocateInode() {

        byte[] memory =
            memoryManager.getFileSystemMemory();

        // TODO:
        // Parcourir les inodes de 0 à MAX_INODES - 1.
        // Identifier le premier inode libre.
        // Retourner son numéro.

        return -1;
    }

    public boolean createFile(
        String directory,
        String filename) {

        int inodeNum = allocateInode();

        if (inodeNum == -1) {
            return false;
        }

        // TODO:
        // Construire l'inode.
        // L'initialiser comme fichier vide.

        return true;
    }

    public MemoryManager getMemoryManager() {
        return memoryManager;
    }
}
```

```
}
}
```

► Tests obligatoires

Étape 9 : Écriture et lecture

Nous pouvons maintenant associer une structure de métadonnées à un fichier. L'étape suivante consiste à placer réellement son contenu dans la mémoire virtuelle. Supposons que l'utilisateur souhaite écrire un tableau :

```
byte[] data
```

de longueur L .

Le VFS utilise des blocs de taille :

$$B = 512$$

Le nombre minimal de blocs nécessaires est :

$$N = \left\lceil \frac{L}{B} \right\rceil$$

Une forme pratique, entièrement entière, est :

$$N = \frac{L + B - 1}{B}$$

avec une division entière.

Par exemple :

$$L = 512 \Rightarrow N = 1$$

alors que :

$$L = 513 \Rightarrow N = 2$$

Le second cas est particulièrement important : un seul octet supplémentaire force l'utilisation d'un second bloc.

Découpage en blocs

Les données doivent être découpées en fragments :

$$chunk_i = data[iB : \min((i+1)B, L)]$$

Chaque fragment est copié dans le bloc physique attribué par le gestionnaire mémoire, notre fameux bitmap d'allocation associé aux pointeur directs et indirects.

L'adresse de début du bloc k est :

$$offset(k) = k \times 512$$

Ainsi, si un fichier utilise les blocs :

```
150
151
```

alors les deux fragments sont respectivement placés à :

```
150 * 512
151 * 512
```

Le VFS doit ensuite enregistrer ces numéros de blocs dans l'inode.

Relation entre inode et contenu

Le contenu du fichier n'est pas nécessairement contigu.

Par exemple, un fichier peut être associé aux blocs :

```
129
140
131
157
```

sans que cela empêche sa lecture. L'inode fournit la correspondance logique :

```
position logique 0..511 -> bloc 129
position logique 512..1023 -> bloc 140
position logique 1024..1535 -> bloc 131
...
```

Cette séparation entre **ordre logique des données** et **position physique des blocs** est précisément ce qui permet au système de fichiers de tolérer la fragmentation.

Un fichier n'a donc pas besoin d'être stocké sous la forme d'une zone physiquement contiguë. Ce sont les métadonnées du fichier qui permettent de reconstruire son contenu logique à partir de blocs physiques éventuellement dispersés.

Limitation des pointeurs directs

Le format fourni contient 10 pointeurs directs. Chaque pointeur désigne un bloc de **512** octets. La capacité maximale directement adressable est donc $10 \times 512 = 5120$ octets. Un fichier nécessitant plus de dix blocs dépasse cette capacité directe.

Le TP conserve toutefois un champ de pointeur indirect dans l'inode. Vous devez respecter les limites explicitement demandées par cette étape : l'implémentation de l'indirection complète n'est pas demandée dans le squelette fourni.

Squelette `writeFile`

```
public boolean writeFile(
    int inodeNum,
    byte[] data) {

    int blocksNeeded =
        (data.length
         + MemoryManager.BLOCK_SIZE - 1)
        / MemoryManager.BLOCK_SIZE;

    if (blocksNeeded > Inode.DIRECT_POINTERS) {
        return false;
    }

    int[] blockPointers =
        new int[Inode.DIRECT_POINTERS];

    // TODO:
    // Allouer blocksNeeded blocs.

    byte[] memory =
        memoryManager.getFileSystemMemory();

    int bytesRemaining =
        data.length;

    int dataSrcOffset = 0;

    // TODO:
    // Pour chaque bloc :
    // - calculer la quantité à copier ;
    // - récupérer le numéro du bloc ;
    // - calculer son offset physique ;
    // - copier les données.

    // TODO:
    // Mettre à jour l'inode.

    return true;
}
```

Squelette `readFile`

```
public byte[] readFile(int inodeNum) {

    Inode inode =
        new Inode(memoryManager, inodeNum);

    int fileSize =
        inode.getFileSize();

    if (fileSize == 0) {
        return new byte[0];
    }

    byte[] fileData =
        new byte[fileSize];

    byte[] memory =
        memoryManager.getFileSystemMemory();

    int[] blockPointers =
        inode.getDirectPointers();

    // TODO:
```

```
// Parcourir les blocs utilisés.
// Copier chaque fragment vers fileData.

return fileData;
}
```

► Tests obligatoires

Fichier exactement égal à un bloc

Ajoutez un test pour un fichier de **512** octets. Le fichier doit nécessiter exactement un bloc.

Vérifiez :

- que l'écriture réussit ;
- que l'inode contient une taille de 512 ;
- qu'un seul pointeur direct est utilisé ;
- que les 512 octets physiques sont corrects ;
- que la lecture restitue exactement les mêmes octets.

Test d'un fichier de 513 octets

Construisez ensuite un tableau de 523 octets.

Vérifiez que :

$$\left\lceil \frac{513}{512} \right\rceil = 2$$

Le test doit notamment contrôler que les 512 premiers octets se trouvent dans le premier bloc et que le dernier octet est placé dans le second bloc.

Vous devez également vérifier les octets directement dans `memory`, et pas seulement le résultat de `readFile`.

Test de dépassement

Construisez un tableau nécessitant plus de dix blocs :

$$L > 10 \times 512$$

L'écriture doit être refusée selon les contraintes du squelette.

Étape 10 : Fragmentation

Supprimer un fichier ne consiste pas uniquement à effacer son nom. Les ressources physiques associées au fichier doivent redevenir disponibles.

Deux catégories d'informations doivent donc être prises en compte :

1. les métadonnées de l'inode ;
2. les blocs de données associés.

La suppression d'un fichier libère les blocs qu'il occupait dans le bitmap. L'inode doit ensuite être considéré comme disponible pour une future allocation. Conceptuellement, si un fichier utilise les blocs `129`, `130`, `140` leur état passe de occupé à libre dans le bitmap. Et ce via la fonction `setBlockUsed` par exemple.

Cette opération est fondamentale car l'espace mémoire du VFS est fini. Une suppression qui ne restituerait pas les blocs provoquerait une fuite de ressources : le système considérerait comme occupés des blocs qui ne sont plus accessibles par aucun fichier.

A savoir

Dans un système de fichiers, il existe deux grands types de fragmentation : externe et interne.

Fragmentation externe : L'espace libre sur le disque (ou en mémoire dans le VFS) finit par être découpé en beaucoup de petits blocs disséminés en fonction des allocations et désallocations. Quand on veut enregistrer un gros fichier, il ne peut pas rentrer dans un seul espace contigu : il doit être réparti dans plusieurs blocs, là où il y a de la place. Cela oblige à découper le fichier en plusieurs morceaux (blocs non contigus) et permet de gérer l'espace efficacement. Mais cause du ralentissement en lecture et écriture.

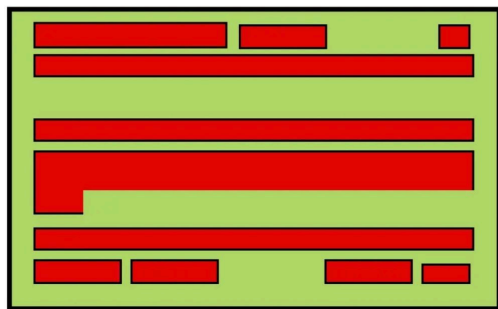
Exemple simplifié : Après plusieurs créations/suppressions on pourrait se retrouver avec ces espaces :

- Bloc 0-129 : occupé par le VFS
- Bloc 130 : occupé
- Bloc 131 : libre
- Bloc 132 : occupé
- Bloc 133 : libre
- Bloc 134 : libre
- Bloc 135 : occupé

Si un fichier de 3 blocs doit être stocké, il aura obligatoirement les blocs 1, 3 et 4.

Fragmentation interne : Cela arrive quand les blocs alloués à un fichier ne sont pas entièrement remplis, par exemple chaque bloc fait 512 octets mais le fichier ne finit pas pile sur cette taille. Le "reste" à la fin du fichier (dans le dernier bloc) est de la mémoire perdue. C'est donc de l'espace gaspillé à l'intérieur d'un bloc réservé et invisible pour le gestionnaire du système de fichiers.

External fragmentation



 Allocated memory
 Free memory

Internal fragmentation



 Allocated memory (block)
 Free memory
 Allocated but Unused memory (os doesn't know about it)

Le VFS ne peut pas "voir" la fragmentation interne (ce qu'il gaspille dans chaque bloc) : il ne peut mesurer que la fragmentation externe (= comment sont répartis les blocs libres/occupés).

Étape 11 : Test avec un fichier externe

Jusqu'à présent, les données utilisées par les tests ont été construites directement dans le programme. Cette stratégie permet de contrôler précisément les cas étudiés, mais elle ne permet pas toujours de détecter les erreurs liées à l'intégration avec une donnée réelle.

Vous allez donc maintenant tester le VFS à partir d'un fichier fourni.

Le fichier constitue une source de données externe au système de fichiers virtuel. Il faut donc réaliser deux opérations conceptuellement distinctes :

1. lire le fichier avec les mécanismes Java ordinaires
2. modifier son contenu dans le VFS

Le test doit ensuite vérifier que les octets récupérés depuis le VFS correspondent aux octets issus du fichier externe.

Le principe peut être représenté ainsi :



Cette étape est particulièrement importante du point de vue logiciel : elle introduit une frontière entre deux systèmes de représentation. Le fichier externe appartient au système de fichiers réel de la machine. Le VFS, lui, possède sa propre organisation interne. Le test vérifie donc que le contenu traverse correctement cette frontière sans corruption.

► Tests obligatoires

Le test externe ne doit pas simplement comparer deux chaînes sans vérifier leur longueur et leurs octets. Le but de cette étape est justement de vérifier que la chaîne traverse correctement les différentes couches de représentation du système.

Étape 12 : Assemblez le `TestRunner`

Le `TestRunner` doit devenir le point d'entrée de la validation.

Votre programme doit appeler successivement les tests des différentes étapes.

```
public static void main(String[] args) {

    testStep2();
    testStep3();
    testStep4();
    testStep5();
    testStep6();
    testStep7();
    testStep8();
    testStep9();
    testStep10();

    if (args.length > 0) {
        testExternalFile(args[0]);
    } else {
        System.out.println(
```

```

        "[INFO] Aucun fichier externe fourni.");
    }

    System.out.println(
        "=== TOUS LES TESTS SONT TERMINÉS ===");
}

```

Compilez :

```
javac *.java
```

Exécutez :

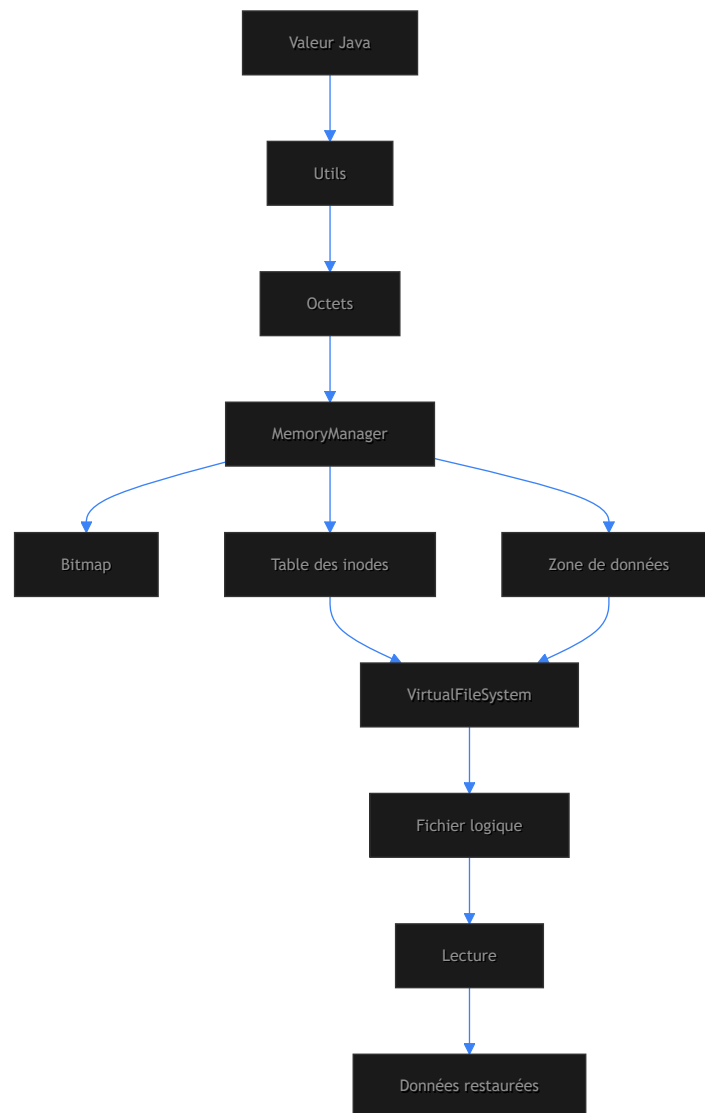
```
java -ea TestRunner
```

Puis, lorsque l'enseignant vous fournit un fichier :

```
java -ea TestRunner fichier.txt
```

Validation globale du système

À ce stade, votre architecture doit pouvoir être comprise comme une chaîne de transformations.



Cette architecture met en évidence une propriété essentielle du système : chaque niveau dépend du précédent, mais ne doit pas nécessairement connaître tous les détails des niveaux supérieurs.

- **Utils** doit savoir transformer des valeurs en octets, mais n'a pas besoin de connaître les fichiers.
- **MemoryManager** doit savoir gérer l'espace mémoire et le bitmap, mais n'a pas besoin de comprendre le contenu sémantique d'un fichier.
 - **Inode** doit connaître la structure des métadonnées d'un fichier et la manière de les localiser dans la mémoire.
 - **VirtualFileSystem** doit utiliser ces mécanismes pour offrir des opérations de plus haut niveau telles que la création, l'écriture et la lecture.

Cette séparation constitue une première approche de l'architecture en couches.

Questions de réflexion

Répondez aux questions suivantes dans votre compte rendu.

Question 1 : Représentation binaire

Pourquoi est-il nécessaire de définir explicitement une convention telle que big-endian lorsqu'une structure est stockée dans un tableau de `byte` ? Que se passerait-il si `writeInt` utilisait big-endian mais `readInt` supposait little-endian sur une autre machine ?

Question 2 : Signes et octets

Pourquoi une opération telle que `memory[offset] & 0xFF` est-elle importante lors de la reconstruction d'une valeur entière ? Expliquez le rôle de l'opérateur binaire en Java.

Question 3 : Sérialisation

Pourquoi tester uniquement `readInt(writeInt(x)) == x` peut-il être insuffisant ? Et si oui comment sampler correctement les valeurs de `x` ? Expliquez en quoi l'inspection directe des octets permet de détecter davantage de classes d'erreurs.

Question 4 : Bitmap

Pourquoi un bitmap est-il plus compact qu'une représentation utilisant un entier par bloc ? Exprimez la taille du bitmap en fonction du nombre N de blocs.

Question 5 : Layout

Pourquoi les structures du système de fichiers doivent-elles occuper des zones mémoire déterministes ? Que se passerait-il si la position du bitmap changeait sans que les méthodes qui l'utilisent soient modifiées ?

Question 6 : Inode

Pourquoi séparer les métadonnées du fichier de son contenu ? Expliquez pourquoi un inode peut être considéré comme une structure permettant de retrouver le contenu d'un fichier sans contenir directement ce contenu.

Question 7 : Allocation

Pourquoi `allocateBlock()` doit-il commencer à rechercher à partir du bloc 129 ? Que se passerait-il si la recherche commençait au bloc 0 ?

Question 8 : Fragmentation

Deux systèmes peuvent-ils posséder exactement le même nombre de blocs libres mais présenter des niveaux de fragmentation différents ? Justifiez votre réponse avec une représentation sous forme de séquence de blocs libres et occupés.

Synthèse architecturale

À la fin du TP, le système obtenu doit être interprétable comme une petite architecture de stockage style zip / archives / vfs. La mémoire brute constitue la couche physique simulée. Le layout définit la partition logique de cette mémoire. Le bitmap fournit l'état d'allocation des blocs. La table des inodes fournit les métadonnées des fichiers. Les pointeurs des inodes établissent la relation entre métadonnées et données physiques.

Le `VirtualFileSystem` fournit finalement une interface de plus haut niveau permettant de manipuler les fichiers.



Syntax error in text
mermaid version 11.17.2

Cette architecture met en évidence une idée générale de l'informatique système : une structure complexe peut être construite progressivement au-dessus d'une représentation très simple, à condition de définir précisément les conventions qui permettent de passer d'un niveau au suivant. C'est aussi une technique de programmation "divide and conquer". Quand le problème global est trop complexe à résoudre : on commence par ce que l'on sait faire et on élague par étapes.

Dans ce TP, ces conventions sont notamment :

valeur Java → octets

puis :

octets → structures binaires

puis :

structures binaires → métadonnées de fichiers

et enfin :

métadonnées + blocs de données → fichier logique

Le système de fichiers virtuel constitue ainsi une application concrète de plusieurs concepts fondamentaux : adressage, représentation binaire, sérialisation, allocation de ressources, structures de données, invariants, abstraction, persistance logique et organisation spatiale de données.

Livraison

La compilation doit fonctionner avec :

```
javac *.java
```

La suite de tests doit être exécutée avec :

```
java -ea TestRunner
```

Le test avec fichier externe doit être exécuté avec :

```
java -ea TestRunner fichier_fourni.txt
```

Un projet qui compile mais dont les tests échouent n'est pas considéré comme terminé. De même, un programme dont les tests passent uniquement parce que les assertions ont été désactivées avec `java TestRunner` ne constitue pas une validation correcte.

La validation attendue porte à la fois sur le comportement observable du VFS et sur la conformité de sa représentation interne : octets, offsets, bitmap, inodes, pointeurs et blocs de données doivent respecter le format défini dans ce TP.